Torneo de Komikan Concurrente

Ailyn Bravo

Asignatura: Sistemas Operativos
Tarea 2 – Threads, Mutexes y Semáforos

1. Diseño del esquema de sincronización

El proyecto implementa un torneo de Komikan concurrente en C++, utilizando hilos y primitivas POSIX para administrar las partidas y proteger los recursos compartidos.

El servidor recibe un total de N jugadores y los empareja de dos en dos. Cada pareja participa en una partida independiente. Para controlar la cantidad máxima de mesas simultáneas, se utiliza un semáforo POSIX (`sem_t`), inicializado con el valor M. Cuando no existen mesas disponibles, las parejas deben esperar hasta que una se libere.

Cada mesa posee su propio tablero, contador de perros capturados, estado de partida y mutex POSIX (`pthread_mutex_t`). Los movimientos se realizan dentro de secciones críticas protegidas mediante `pthread_mutex_lock()` y

`pthread_mutex_unlock()`, evitando que varios hilos modifiquen simultáneamente el mismo tablero.

Para el emparejamiento se utiliza una cola de jugadores protegida por un mutex del torneo. Además, se utilizan variables de condición (`pthread_cond_t`) para coordinar la espera y notificación de los jugadores cuando se completa una pareja.

2. Análisis de riesgos de concurrencia

Durante el desarrollo se identificaron los siguientes riesgos:

Deadlock: puede producirse cuando un hilo mantiene un mutex bloqueado e intenta acceder nuevamente al mismo recurso. Para prevenirlo, se procura liberar los mutex después de cada operación crítica y evitar bloqueos innecesarios.

Livelock: puede ocurrir cuando varios hilos realizan intentos de movimiento que interfieren entre sí sin conseguir avanzar. Para disminuir esta posibilidad, se utilizan retardos aleatorios entre 50 y 300 milisegundos, distribuyendo los intentos de movimiento.

Starvation: puede presentarse cuando un hilo o una pareja permanece esperando demasiado tiempo por un recurso disponible. El semáforo administra el acceso a las mesas y las variables de condición coordinan la espera de los jugadores. Sin embargo, no se garantiza un orden estricto de atención.

3. Evaluación y decisiones de diseño

Se eligió una arquitectura basada en hilos porque permite representar de manera independiente los movimientos del Puma y los doce perros, respetando la naturaleza concurrente del juego.

El uso de un mutex por mesa permite mantener la consistencia del tablero sin bloquear las partidas que se ejecutan en otras mesas. Por su parte, el semáforo limita la cantidad de partidas simultáneas y permite controlar el uso de recursos del sistema.

Los retardos aleatorios permiten simular una competencia en tiempo real, sin turnos estrictos. Esta decisión favorece la concurrencia, aunque puede generar competencia por las mismas posiciones y una latencia variable en los movimientos.

Para la interrupción del servidor se contempla el uso de `SIGINT` mediante `sigwait()`, junto con una variable de detención compartida. Esto permite coordinar la finalización de las partidas y los hilos.

Como aspectos pendientes de validación se consideran el manejo completo de fallos o abandonos de jugadores y la comprobación de la liberación de todos los recursos POSIX ante una interrupción.